Département GI — Mod. SSI-AT
UAE-ENSATé — 2026

Documentation Technique pour le Membre 2

Mini-Projet : **ChriOnline**

Application E-Commerce — Architecture Sockets TCP/UDP

Rédigé par : Mohamed Amine Ait Jaakike

Tâches concernées : T05, T06, T07 (Membre 2)
Tâches dépendantes : T08, T09, T10 (Mohamed Amine Ait Jaakike)

Mars 2026

Table des matières

1 Contexte et Objectif de ce Document **2**

2 Répartition des Tâches **2**

3 Ce que Membre 3 a Implémenté **2**
3.1 T08 — Endpoints Panier (ADD / REMOVE / GET_CART / CLEAR_CART) 2
3.1.1 Fichiers créés . . . 2
3.1.2 Détail des méthodes . . . 2
3.2 T09 — Commande + Paiement Simulé . . . 3
3.2.1 Fichiers créés . . . 3
3.2.2 Flux de `PLACE_ORDER` . . . 3
3.2.3 Paiement simulé . . . 4
3.3 T10 — Logger Serveur . . . 4
3.3.1 Fichier créé . . . 4
3.3.2 Caractéristiques . . . 4

4 Contrats et Interfaces à Respecter **4**
4.1 Format des Requêtes / Réponses . . . 4
4.2 Fichiers existants à NE PAS modifier . . . 5
4.3 Fichiers à compléter . . . 5

5 Guide d'Implémentation — T05 : Authentification **5**
5.1 Étape 1 : Compléter UserDAO . . . 5
5.2 Étape 2 : Gestion des Sessions . . . 7
5.3 Étape 3 : Compléter AuthService . . . 8

6 Guide d'Implémentation — T06 : Endpoints Produits **10**
6.1 Étape 1 : Compléter ProductDAO . . . 10
6.2 Étape 2 : Compléter ProductService . . . 11

7 Guide d'Implémentation — T07 : LOGOUT + Expiration **13**

8 Wiring dans RequestRouter **13**

9 Intégration Token → userId (sans modifier Membre 3) **14**

10 Comment Tester **15**
10.1 Méthode 1 : MainTest (sans réseau) . . . 15
10.2 Méthode 2 : Telnet (avec réseau) . . . 16

11 Résumé des Fichiers à Créer / Modifier **16**

12 Règles d'Or **16**

1 Contexte et Objectif de ce Document

Ce document est destiné au **Membre 2** de l'équipe ChriOnline. Il décrit :

1. Ce que le Membre 3 a déjà implémenté (T08, T09, T10).
2. Les **interfaces et contrats** que les tâches du Membre 2 (T05, T06, T07) doivent respecter pour s'intégrer sans conflit.
3. Un **guide pas-à-pas** pour implémenter chaque tâche.
4. Comment **tester** chaque endpoint.

2 Répartition des Tâches

ID	Tâche	Membre	Priorité	Jours
T01	Init projet Git + Maven	Membre 1	High	0.5
T02	Schéma SQL + seed data	Membre 1	High	1
T03	Serveur TCP multi-clients	Membre 1	High	1.5
T04	Protocole JSON (Request/Response)	Membre 1	High	1
T05	Auth : REGISTER + LOGIN + token	**Membre 2**	High	1.5
T06	Endpoints produits	**Membre 2**	High	2
T07	LOGOUT + expiration sessions	**Membre 2**	High	1
T08	Endpoints panier	Membre 3	High	1.5
T09	Commande + paiement simulé	Membre 3	High	1.5
T10	Logger serveur	Membre 3	High	1

TABLE 1 – Répartition complète des tâches

3 Ce que Membre 3 a Implémenté

3.1 T08 — Endpoints Panier (ADD / REMOVE / GET_CART / CLEAR_CART)

3.1.1 Fichiers créés

— `server/dao/CartDAO.java` — 7 méthodes JDBC

— `server/service/CartService.java` — 4 méthodes métier

— Wiring dans `RequestRouter` (cases `ADD_TO_CART`, `REMOVE_FROM_CART`, `GET_CART`, `CLEAR_CART`)

3.1.2 Détail des méthodes

CartDAO

`tIdByUserId(userId)` Retourne le `cart_id` pour un utilisateur, ou `-1` si aucun panier n'existe.

`createCart(userId)` Insère un nouveau panier et retourne le `cart_id` généré.

`etCartItems(cartId)` JOIN `cart_items` + `products`. Retourne une liste de maps contenant : `cartItemId`, `cartId`, `productId`, `productName`, `productDescription`, `productPrice`, `stockQuantit` `imageUrl`, `quantity`, `unitPrice`, `subtotal`.

`(cartId, productId)`	Vérifie si un produit est déjà dans le panier.
`Id, qty, unitPrice)`	Si le produit existe déjà → UPDATE `quantity += qty`. Sinon → INSERT.
`(cartId, productId)`	DELETE l'item. Retourne `true` si une ligne a été supprimée.
`clearCart(cartId)`	DELETE tous les items du panier.

CartService

`getCart(request)`	Lit `data.userId` → récupère les items → calcule le total → retourne `{cartId, items, total, itemCount}`.
`addToCart(request)`	Lit `data.userId, data.productId, data.quantity`. Vérifie via **ProductDAO.getProduc** que le produit existe, est disponible et a du stock suffisant. Crée le panier si nécessaire. Ajoute l'item.
`veFromCart(request)`	Lit `data.userId, data.productId`. Supprime l'item du panier.
`clearCart(request)`	Lit `data.userId`. Vide le panier.

`CartService.addToCart()` utilise `ProductDAO.getProductById()` pour vérifier le produit. Ce DAO existe déjà (créé par Membre 3 dans T09). Le Membre 2 doit **compléter** `ProductDAO` avec de nouvelles méthodes (`getAllProducts`, `searchProducts`) **sans modifier ni supprimer** `getProductById()` ni `updateStock()`.

3.2 T09 — Commande + Paiement Simulé

3.2.1 Fichiers créés

- `server/dao/OrderDAO.java` — 6 méthodes
- `server/dao/PaymentDAO.java` — 3 méthodes
- `server/dao/ProductDAO.java` — 2 méthodes (`getProductById`, `updateStock`)
- `server/service/OrderService.java` — 3 endpoints
- `server/service/PaymentService.java` — 1 endpoint

3.2.2 Flux de `PLACE_ORDER`

1. Valider les champs obligatoires (`shippingAddress`, `paymentMethod`).
2. Récupérer le panier via `CartDAO`.
3. Vérifier le stock de chaque item via `ProductDAO`.
4. Créer la commande avec ID unique `ORD-XXXXXXXX` (UUID).
5. Créer les `order_items` avec un `product_snapshot` (JSON figé).
6. Déduire le stock via `ProductDAO.updateStock()`.
7. Simuler le paiement (90 % accepté, 10 % refusé).
8. Si accepté → statut commande = `VALIDEE`.
9. Vider le panier.
10. Retourner le résultat complet.

3.2.3 Paiement simulé

- IDs générés : `PAY-XXXXXXXX`, `TXN-XXXXXXXX`.
- `PROCESS_PAYMENT` permet de retenter un paiement refusé.
- Un paiement déjà `ACCEPTE` ne peut pas être retraité.

3.3 T10 — Logger Serveur

3.3.1 Fichier créé

`server/core/ServerLogger.java`

3.3.2 Caractéristiques

- Dual output : console (`System.out`) + fichier (`server.log`).
- Format : `[yyyy-MM-dd HH:mm:ss.SSS] [LEVEL] [thread] message`.
- Thread-safe (`synchronized`).
- 3 niveaux : `INFO`, `WARN`, `ERROR`.
- Intégré dans : `ClientHandler`, `RequestRouter`, `CartService`, `OrderService`, `PaymentService`.

Le Membre 2 doit utiliser `ServerLogger.info()`, `.warn()`, `.error()` dans ses services (AuthService, ProductService) pour maintenir la cohérence de la journalisation.

4 Contrats et Interfaces à Respecter

4.1 Format des Requêtes / Réponses

Request (client → serveur) :

```
{
    "action": "LOGIN",
    "token": null,
    "data": {
        "username": "alice",
        "password": "secret123"
    }
}
```

Response (serveur → client) :

```
{
    "success": true,
    "message": "Login successful",
    "data": {
        "token": "abc-123-xyz",
        "userId": 2,
        "username": "alice",
        "role": "CLIENT"
    }
}
```

4.2 Fichiers existants à NE PAS modifier

Fichier	Raison
`CartDAO.java`	Utilisé par CartService (T08)
`CartService.java`	Endpoints panier fonctionnels
`OrderDAO.java`	Utilisé par OrderService (T09)
`OrderService.java`	Endpoints commandes fonctionnels
`PaymentDAO.java`	Utilisé par PaymentService (T09)
`PaymentService.java`	Endpoint paiement fonctionnel
`ServerLogger.java`	Utilisé partout (T10)
`Request.java`	Protocole partagé (T04)
`Response.java`	Protocole partagé (T04)
`Action.java`	Enum partagée (T04)

TABLE 2 – Fichiers protégés — ne pas modifier

4.3 Fichiers à compléter

Fichier	Action requise
`UserDAO.java`	Actuellement vide. Créer toutes les méthodes.
`AuthService.java`	Actuellement vide. Implémenter LOGIN/REGISTER/LOGOUT.
`ProductService.java`	Actuellement vide. Implémenter GET_PRODUCTS / GET_DETAIL / SEARCH.
`ProductDAO.java`	**Ajouter** des méthodes (`getAllProducts`, `searchProducts`). Ne PAS modifier `getProductById` ni `updateStock`.
`RequestRouter.java`	Remplacer les placeholders `handleAuth` et `handleProducts`.

TABLE 3 – Fichiers à compléter par Membre 2

5 Guide d'Implémentation — T05 : Authentification

5.1 Étape 1 : Compléter UserDAO

Listing 1 – UserDAO.java — méthodes à implémenter

```
1 package com.chrionline.server.dao;
2
3 import java.sql.*;
4 import java.util.HashMap;
5 import java.util.Map;
6
```

```java
public class UserDAO {

    private Connection conn() {
        return DBConnection.getInstance().getConnection();
    }

    public Map<String, Object> findByUsername(String username)
            throws SQLException {
        String sql = "SELECT * FROM users WHERE username = ?";
        try (PreparedStatement ps = conn().prepareStatement(sql)) {
            ps.setString(1, username);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return mapUser(rs);
            return null;
        }
    }

    public Map<String, Object> findByEmail(String email)
            throws SQLException {
        String sql = "SELECT * FROM users WHERE email = ?";
        try (PreparedStatement ps = conn().prepareStatement(sql)) {
            ps.setString(1, email);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return mapUser(rs);
            return null;
        }
    }

    public Map<String, Object> findById(int userId)
            throws SQLException {
        String sql = "SELECT * FROM users WHERE user_id = ?";
        try (PreparedStatement ps = conn().prepareStatement(sql)) {
            ps.setInt(1, userId);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return mapUser(rs);
            return null;
        }
    }

    public int createUser(String username, String email,
            String passwordHash, String firstName,
            String lastName, String address, String phone)
            throws SQLException {
        String sql = "INSERT INTO users (username, email, "
            + "password_hash, first_name, last_name, address, "
            + "phone) VALUES (?, ?, ?, ?, ?, ?, ?)";
        try (PreparedStatement ps = conn().prepareStatement(sql,
                Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, username);
            ps.setString(2, email);
            ps.setString(3, passwordHash);
            ps.setString(4, firstName);
            ps.setString(5, lastName);
            ps.setString(6, address);
            ps.setString(7, phone);
            ps.executeUpdate();
            ResultSet rs = ps.getGeneratedKeys();
            if (rs.next()) return rs.getInt(1);
            return -1;
```

```
        }
    }

    private Map<String, Object> mapUser(ResultSet rs)
            throws SQLException {
        Map<String, Object> u = new HashMap<>();
        u.put("userId", rs.getInt("user_id"));
        u.put("username", rs.getString("username"));
        u.put("email", rs.getString("email"));
        u.put("passwordHash", rs.getString("password_hash"));
        u.put("firstName", rs.getString("first_name"));
        u.put("lastName", rs.getString("last_name"));
        u.put("address", rs.getString("address"));
        u.put("phone", rs.getString("phone"));
        u.put("role", rs.getString("role"));
        u.put("isActive", rs.getBoolean("is_active"));
        return u;
    }
}
```

5.2 Étape 2 : Gestion des Sessions

Créer une classe pour gérer les sessions en mémoire :

Listing 2 – SessionManager.java — gestion des tokens

```
package com.chrionline.server.service;

import com.chrionline.server.core.ServerLogger;

import java.time.LocalDateTime;
import java.util.Map;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;

public class SessionManager {

    private static final int SESSION_DURATION_MINUTES = 30;

    private static final Map<String, SessionData> sessions
            = new ConcurrentHashMap<>();

    public static String createSession(int userId, String role) {
        String token = UUID.randomUUID().toString();
        sessions.put(token, new SessionData(
                userId, role, LocalDateTime.now(),
                LocalDateTime.now().plusMinutes(
                        SESSION_DURATION_MINUTES)));
        ServerLogger.info("Session created: token="
                + token + " userId=" + userId);
        return token;
    }

    public static SessionData getSession(String token) {
        if (token == null) return null;
        SessionData sd = sessions.get(token);
        if (sd == null) return null;
        if (LocalDateTime.now().isAfter(sd.expiresAt)) {
            sessions.remove(token);
```

```
            ServerLogger.info("Session expired: " + token);
            return null;
        }
        return sd;
    }

    public static boolean invalidate(String token) {
        SessionData removed = sessions.remove(token);
        if (removed != null) {
            ServerLogger.info("Session invalidated: " + token);
            return true;
        }
        return false;
    }

    public static void cleanExpired() {
        sessions.entrySet().removeIf(e ->
                LocalDateTime.now().isAfter(e.getValue().expiresAt));
    }

    public static class SessionData {
        public final int userId;
        public final String role;
        public final LocalDateTime createdAt;
        public final LocalDateTime expiresAt;

        public SessionData(int userId, String role,
                LocalDateTime createdAt,
                LocalDateTime expiresAt) {
            this.userId = userId;
            this.role = role;
            this.createdAt = createdAt;
            this.expiresAt = expiresAt;
        }
    }
}
```

5.3 Étape 3 : Compléter AuthService

Listing 3 – AuthService.java

```
package com.chrionline.server.service;

import com.chrionline.server.core.ServerLogger;
import com.chrionline.server.dao.UserDAO;
import com.chrionline.shared.dto.Request;
import com.chrionline.shared.dto.Response;

import java.util.HashMap;
import java.util.Map;

public class AuthService {

    private final UserDAO userDAO = new UserDAO();

    public Response register(Request request) {
        try {
            String username = request.getString("username");
```

```
            String email = request.getString("email");
            String password = request.getString("password");
            String firstName = request.getString("firstName");
            String lastName = request.getString("lastName");
            String address = request.getString("address");
            String phone = request.getString("phone");

            if (username == null || password == null
                    || email == null)
                return Response.error(
                        "username, email, password required");

            if (userDAO.findByUsername(username) != null)
                return Response.error("Username already exists");
            if (userDAO.findByEmail(email) != null)
                return Response.error("Email already exists");

            // Hash simple (remplacer par bcrypt si besoin)
            String hash = Integer.toHexString(
                    password.hashCode());

            int userId = userDAO.createUser(username, email,
                    hash, firstName, lastName, address, phone);

            ServerLogger.info("REGISTER userId=" + userId
                    + " username=" + username);
            return Response.ok("Registration successful",
                    Map.of("userId", userId,
                           "username", username));
        } catch (Exception e) {
            ServerLogger.error("register: " + e.getMessage());
            return Response.error("Registration failed: "
                    + e.getMessage());
        }
    }

    public Response login(Request request) {
        try {
            String username = request.getString("username");
            String password = request.getString("password");

            if (username == null || password == null)
                return Response.error(
                        "username and password required");

            Map<String, Object> user =
                    userDAO.findByUsername(username);
            if (user == null)
                return Response.error("User not found");

            String hash = Integer.toHexString(
                    password.hashCode());
            if (!hash.equals(user.get("passwordHash")))
                return Response.error("Invalid password");

            if (!(boolean) user.get("isActive"))
                return Response.error("Account disabled");

            String token = SessionManager.createSession(
```

```
                    (int) user.get("userId"),
                    (String) user.get("role"));

            Map<String, Object> data = new HashMap<>();
            data.put("token", token);
            data.put("userId", user.get("userId"));
            data.put("username", user.get("username"));
            data.put("role", user.get("role"));

            ServerLogger.info("LOGIN userId="
                    + user.get("userId"));
            return Response.ok("Login successful", data);
        } catch (Exception e) {
            ServerLogger.error("login: " + e.getMessage());
            return Response.error("Login failed: "
                    + e.getMessage());
        }
    }

    public Response logout(Request request) {
        try {
            String token = request.getToken();
            if (token == null)
                return Response.error("Token required");

            boolean ok = SessionManager.invalidate(token);
            if (!ok) return Response.error("Invalid token");

            ServerLogger.info("LOGOUT token=" + token);
            return Response.ok("Logged out", null);
        } catch (Exception e) {
            ServerLogger.error("logout: " + e.getMessage());
            return Response.error("Logout failed: "
                    + e.getMessage());
        }
    }
}
```

6 Guide d'Implémentation — T06 : Endpoints Produits

6.1 Étape 1 : Compléter ProductDAO

`ProductDAO` contient déjà deux méthodes créées par Membre 3 :

- `getProductById(int productId)` — utilisée par CartService et OrderService
- `updateStock(int productId, int qty)` — utilisée par OrderService

Ne pas les modifier ni les supprimer. Ajouter les nouvelles méthodes en dessous.

Listing 4 – Méthodes à AJOUTER dans ProductDAO.java

```
// === AJOUTER ces methodes dans ProductDAO.java ===
// === NE PAS toucher getProductById ni updateStock ===

public List<Map<String, Object>> getAllProducts()
```

```
        throws SQLException {
    String sql = "SELECT p.*, c.name AS category_name "
        + "FROM products p "
        + "LEFT JOIN categories c "
        + "ON p.category_id = c.category_id "
        + "WHERE p.is_available = true "
        + "ORDER BY p.product_id";
    List<Map<String, Object>> list = new ArrayList<>();
    try (PreparedStatement ps = conn().prepareStatement(sql)) {
        ResultSet rs = ps.executeQuery();
        while (rs.next()) list.add(mapProduct(rs));
    }
    return list;
}

public List<Map<String, Object>> searchProducts(String keyword)
        throws SQLException {
    String sql = "SELECT p.*, c.name AS category_name "
        + "FROM products p "
        + "LEFT JOIN categories c "
        + "ON p.category_id = c.category_id "
        + "WHERE p.is_available = true "
        + "AND (p.name LIKE ? OR p.description LIKE ?) "
        + "ORDER BY p.product_id";
    List<Map<String, Object>> list = new ArrayList<>();
    try (PreparedStatement ps = conn().prepareStatement(sql)) {
        String pattern = "%" + keyword + "%";
        ps.setString(1, pattern);
        ps.setString(2, pattern);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) list.add(mapProduct(rs));
    }
    return list;
}

private Map<String, Object> mapProduct(ResultSet rs)
        throws SQLException {
    Map<String, Object> p = new HashMap<>();
    p.put("productId", rs.getInt("product_id"));
    p.put("name", rs.getString("name"));
    p.put("description", rs.getString("description"));
    p.put("price", rs.getDouble("price"));
    p.put("stockQuantity", rs.getInt("stock_quantity"));
    p.put("categoryId", rs.getInt("category_id"));
    p.put("categoryName", rs.getString("category_name"));
    p.put("imageUrl", rs.getString("image_url"));
    p.put("isAvailable", rs.getBoolean("is_available"));
    return p;
}
```

Imports à ajouter en haut de ProductDAO :

```
import java.util.ArrayList;
import java.util.List;
```

6.2 Étape 2 : Compléter ProductService

Listing 5 – ProductService.java

```
package com.chrionline.server.service;

import com.chrionline.server.core.ServerLogger;
import com.chrionline.server.dao.ProductDAO;
import com.chrionline.shared.dto.Request;
import com.chrionline.shared.dto.Response;

import java.util.List;
import java.util.Map;

public class ProductService {

    private final ProductDAO productDAO = new ProductDAO();

    public Response getProducts(Request request) {
        try {
            List<Map<String, Object>> products =
                    productDAO.getAllProducts();
            ServerLogger.info("GET_PRODUCTS count="
                    + products.size());
            return Response.ok(products);
        } catch (Exception e) {
            ServerLogger.error("getProducts: "
                    + e.getMessage());
            return Response.error("Failed to get products: "
                    + e.getMessage());
        }
    }

    public Response getProductDetail(Request request) {
        try {
            int productId = request.getInt("productId");
            Map<String, Object> product =
                    productDAO.getProductById(productId);
            if (product == null)
                return Response.error("Product not found");
            ServerLogger.info("GET_PRODUCT_DETAIL id="
                    + productId);
            return Response.ok(product);
        } catch (Exception e) {
            ServerLogger.error("getProductDetail: "
                    + e.getMessage());
            return Response.error("Failed: " + e.getMessage());
        }
    }

    public Response searchProducts(Request request) {
        try {
            String keyword = request.getString("keyword");
            if (keyword == null || keyword.isEmpty())
                return Response.error("Keyword required");
            List<Map<String, Object>> results =
                    productDAO.searchProducts(keyword);
            ServerLogger.info("SEARCH_PRODUCTS keyword='"
                    + keyword + "' results="
                    + results.size());
            return Response.ok(results);
        } catch (Exception e) {
            ServerLogger.error("searchProducts: "
```

```
                    + e.getMessage());
            return Response.error("Failed: " + e.getMessage());
        }
    }
}
```

7 Guide d'Implémentation — T07 : LOGOUT + Expiration

Le LOGOUT est déjà inclus dans `AuthService.logout()`. L'expiration automatique est gérée par `SessionManager.getSession()` qui vérifie `expiresAt` à chaque appel.

Pour un nettoyage périodique, ajouter dans `ServerMain` :

Listing 6 – Nettoyage périodique dans ServerMain

```
// Ajouter dans ServerMain.main(), AVANT server.start()
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

Executors.newSingleThreadScheduledExecutor()
    .scheduleAtFixedRate(
        SessionManager::cleanExpired,
        5, 5, TimeUnit.MINUTES);
```

8 Wiring dans RequestRouter

Le Membre 2 doit remplacer les placeholders dans `RequestRouter`. Voici les modifications exactes :

Listing 7 – Modifications dans RequestRouter.java

```
// 1. Ajouter les imports
import com.chrionline.server.service.AuthService;
import com.chrionline.server.service.ProductService;

// 2. Ajouter les instances (a cote des existants)
private static final AuthService authService
        = new AuthService();
private static final ProductService productService
        = new ProductService();

// 3. Remplacer handleAuth
private static Response handleAuth(Request request) {
    switch (request.getAction()) {
        case REGISTER:
            return authService.register(request);
        case LOGIN:
            return authService.login(request);
        case LOGOUT:
            return authService.logout(request);
        default:
            return Response.error("Unknown auth action");
    }
}

```

```java
// 4. Remplacer handleProducts
private static Response handleProducts(Request request) {
    switch (request.getAction()) {
        case GET_PRODUCTS:
            return productService.getProducts(request);
        case GET_PRODUCT_DETAIL:
            return productService.getProductDetail(request);
        case SEARCH_PRODUCTS:
            return productService.searchProducts(request);
        default:
            return Response.error("Unknown product action");
    }
}
```

Ne pas toucher aux blocs `case ADD_TO_CART`, `case PLACE_ORDER`, `case PROCESS_PAYMENT`, etc. Ils sont déjà câblés par Membre 3 et fonctionnels.

9 Intégration Token → userId (sans modifier Membre 3)

Actuellement, les services du Membre 3 lisent `request.getInt("userId")` directement. Pour que l'authentification fonctionne sans modifier le code du Membre 3, le Membre 2 doit ajouter une résolution de token dans `RequestRouter`, **avant** d'appeler les services.

Listing 8 – Helper à ajouter dans RequestRouter

```java
/**
 * Resout le token en userId et l'injecte dans data.
 * Retourne null si OK, ou une Response d'erreur.
 */
private static Response resolveToken(Request request) {
    String token = request.getToken();
    if (token == null)
        return Response.error("Authentication required");

    SessionManager.SessionData sd =
            SessionManager.getSession(token);
    if (sd == null)
        return Response.error("Invalid or expired token");

    // Injecter userId dans data pour les services
    if (request.getData() == null)
        request.setData(new java.util.HashMap<>());
    request.getData().put("userId", sd.userId);
    return null; // OK
}
```

Listing 9 – Utilisation dans route() — exemple pour Cart

```java
// Dans le switch, AVANT d'appeler les services :
case ADD_TO_CART:
case REMOVE_FROM_CART:
case GET_CART:
case CLEAR_CART: {
    Response authCheck = resolveToken(request);
    if (authCheck != null) return authCheck;
```

```
    // Puis dispatcher normalement
    switch (request.getAction()) {
        case ADD_TO_CART:
            return cartService.addToCart(request);
        case REMOVE_FROM_CART:
            return cartService.removeFromCart(request);
        case GET_CART:
            return cartService.getCart(request);
        case CLEAR_CART:
            return cartService.clearCart(request);
    }
}
```

Grâce à cette approche, les services du Membre 3 continuent à lire `request.getInt("userId")` sans aucune modification. Le token est résolu en `userId` et injecté dans `data` par le routeur.

10 Comment Tester

10.1 Méthode 1 : MainTest (sans réseau)

Listing 10 – Tests pour T05/T06

```
// === T05 : Auth ===
test("REGISTER",
    new Request(Action.REGISTER, null, Map.of(
        "username", "testuser",
        "email", "test@test.com",
        "password", "pass123",
        "firstName", "Test",
        "lastName", "User",
        "address", "1 Rue Test",
        "phone", "0600000000")));

test("LOGIN",
    new Request(Action.LOGIN, null, Map.of(
        "username", "testuser",
        "password", "pass123")));
// -> Recuperer le token de la reponse

test("LOGIN mauvais mdp",
    new Request(Action.LOGIN, null, Map.of(
        "username", "testuser",
        "password", "wrong")));

// === T06 : Produits ===
test("GET_PRODUCTS",
    new Request(Action.GET_PRODUCTS, null, null));

test("GET_PRODUCT_DETAIL",
    new Request(Action.GET_PRODUCT_DETAIL, null,
        Map.of("productId", 1)));

test("SEARCH_PRODUCTS",
```

```
32      new Request(Action.SEARCH_PRODUCTS, null,
33          Map.of("keyword", "phone")));
34
35  // === T07 : Logout ===
36  test("LOGOUT",
37      new Request(Action.LOGOUT, "token-ici", null));
```

10.2 Méthode 2 : Telnet (avec réseau)

1. Lancer `ServerMain` (port 9090)
2. Ouvrir un terminal : `telnet localhost 9090`
3. Envoyer des requêtes JSON (une par ligne) :

Listing 11 – Requêtes telnet

```
{"action":"REGISTER","token":null,"data":{"username":"bob","email":"
    bob@mail.com","password":"1234","firstName":"Bob","lastName":"Test","
    address":"Paris","phone":"06"}}
{"action":"LOGIN","token":null,"data":{"username":"bob","password
    ":"1234"}}
{"action":"GET_PRODUCTS","token":null,"data":null}
{"action":"GET_PRODUCT_DETAIL","token":null,"data":{"productId":1}}
{"action":"SEARCH_PRODUCTS","token":null,"data":{"keyword":"phone"}}
```

11 Résumé des Fichiers à Créer / Modifier

Tâche	Fichier	Action	Package
T05	`UserDAO.java`	Compléter	`server.dao`
T05	`SessionManager.java`	Créer	`server.service`
T05	`AuthService.java`	Compléter	`server.service`
T06	`ProductDAO.java`	Ajouter méthodes	`server.dao`
T06	`ProductService.java`	Compléter	`server.service`
T07	(inclus dans T05)	—	—
—	`RequestRouter.java`	Modifier placeholders	`server.core`

TABLE 4 – Checklist des fichiers pour Membre 2

12 Règles d'Or

1. **Utiliser ServerLogger** dans tous les services et DAOs pour la cohérence du logging.
2. **Ne pas modifier** les fichiers listés dans la section 4 (table des fichiers protégés).
3. **Ajouter** les méthodes dans `ProductDAO` sans toucher à `getProductById` ni `updateStock`.
4. **Suivre le même pattern** que les services existants : `try/catch` → `ServerLogger` → `Response.ok/error`.

5. **Résoudre le token** dans `RequestRouter` (section 9) pour ne pas impacter le code du Membre 3.
6. **Tester** chaque endpoint individuellement avant de pousser sur Git.